

STAT / ML / CODE

# Course Overview & Software Setup

STAT3009 · Recommender Systems

Ben Dai

CUHK · Department of Statistics and Data Science



# Every platform chooses what to show next

## 01 · COURSE PURPOSE

| products | music | connections | films | investments |
|----------|-------|-------------|-------|-------------|
|----------|-------|-------------|-------|-------------|

AMAZON · ALIBABA · NETFLIX · SPOTIFY · LINKEDIN · FINANCE

Recommendation turns many possible choices into a useful next decision.



# The semester moves from baselines to neural recommenders

## COURSE MAP

### FOUNDATIONS

#### Frame the problem

Interaction data  
Evaluation  
Mean baselines

### LATENT STRUCTURE

#### Learn hidden preferences

Cross-validation  
SVD and matrix factorization  
MovieLens

### NEURAL RS

#### Add expressive models

Neural networks  
Neural matrix factorization  
Side information

STATISTICS × MACHINE LEARNING × CODE



# How this course works

Understand the expectations before we begin the technical work.



02



# You will learn, build, and compete

01

## LEARN

Concepts in slides and notes

02

## BUILD

Live Python in Colab and Jupyter

03

## COMPETE

In-class practice and Kaggle

[Course homepage ↗](#) · [GitHub repository ↗](#)



# Bring foundations—not prior recommender-system experience

## WHAT HELPS

### We build from tools you have already seen

You do not need to arrive as a recommender-systems expert.

Python-library tutorials are provided when we need them.

**01**

#### Statistics

Linear and ridge regression; hypothesis testing

**02**

#### Programming

Python, NumPy, Pandas, basic scikit-learn

**03**

#### Mathematics

Linear algebra, calculus, probability



# Assessment rewards implementation

15%

Homework

40%

Open-book  
in-class Kaggle

Approximately mid-semester

45%

Final in-class  
coding quiz

Last class of the semester



# Use AI as a tutor—not a substitute for understanding

AI-ASSISTED · STUDENT-OWNED

## USE IT TO

### Question · compare · debug

Ask for an explanation of an error, compare two approaches, or improve the clarity of your Markdown.

## VERIFY IT

### Run · inspect · test

Execute every cell, read the output, check assumptions, and test the suggestion on the actual data.

## OWN IT

### Explain · modify · reproduce

You remain responsible for every submitted result and should be able to adapt the code yourself.

**Not evidence of learning:** submitting generated work that you cannot reproduce, justify, or debug.

Follow each assessment's stated tool and disclosure rules. During an assessment, use only the tools explicitly permitted.



# Building a knowledge landscape

Know where an idea fits, what it depends on, and what you need to learn next.

| Foundation                                                    | Connection in this course                                       | A question you can ask                              |
|---------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| <b>Linear algebra</b><br>Vectors and inner products           | Matrix factorization<br>Represents users and items with vectors | What does the predicted score represent?            |
| <b>Probability and statistics</b><br>Sampling and uncertainty | Model evaluation<br>Uses held-out data as evidence              | Will this result extend to new ratings?             |
| <b>Coding</b><br>Arrays, indexing, functions                  | Computational experiments<br>Turns a model into data operations | Does the program implement the intended experiment? |

A model needs a representation, an implementation, and evidence about its predictions.

Use this map to locate an AI suggestion and identify the knowledge you need to use it.



# Mathematics and coding in the AI era

Mathematics and coding turn your knowledge landscape into something you can reason with and use.

---

## Mathematics

Define the problem and judge the evidence.

- Connect models to their assumptions and objectives.
- Understand what an evaluation result supports.

Does a lower rating error imply a better recommendation list?

---

## Coding

Understand and adapt the computation.

- Trace how a modeling idea becomes an experiment.
- Modify the workflow when data or requirements change.

Which observations went into fitting and evaluation?

---

Use these foundations to guide your questions and extend your understanding with AI.



# Performance on familiar questions

A student can solve every question on the practice sheet.

What would convince you that they can solve new questions?

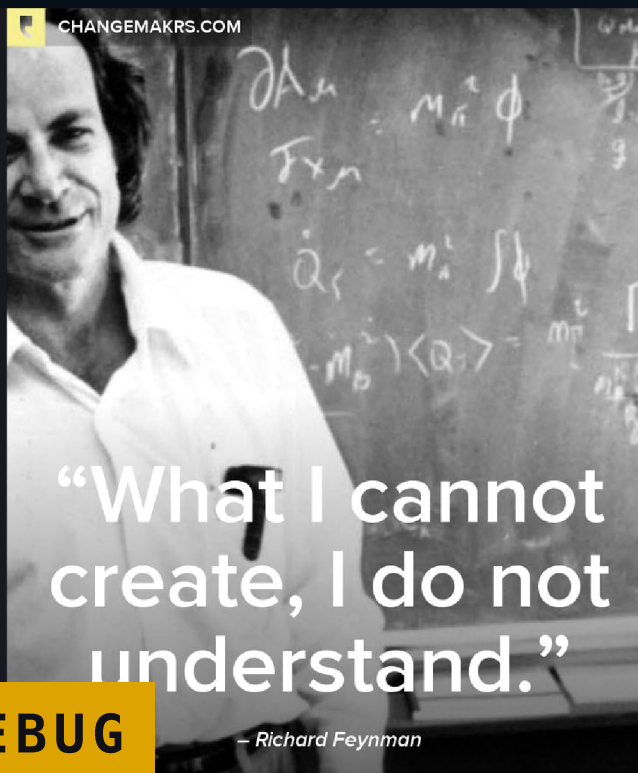
---

Try a separate set of questions that was not used for practice.

# The best way to learn code is to wrestle with it

PRACTICE  
MAKES  
PERFECT.

PRACTICE



DEBUG

“What I cannot  
create, I do not  
understand.”

— Richard Feynman

be  
PATIENT

PERSIST

**Bring a laptop.** Read code, run it, break it, fix it—and make it yours.

# Software preparation

Use notebooks to keep executable code and readable reasoning together.



03



# Three layers make a Colab notebook work

## ONE COMPUTATIONAL WORKFLOW

### LANGUAGE

## Python

The instructions and objects we write.

LIVES  
INSIDE

### DOCUMENT

## Jupyter Notebook

An `.ipynb` file containing code, text, and saved output.

RUNS  
THROUGH

### SERVICE

## Google Colab

A hosted notebook interface and computational runtime.

Colab is based on Jupyter; Python is the primary language executed by its kernel.



# A notebook mixes Python and Markdown

## JUPYTER NOTEBOOK

### PYTHON

## Compute

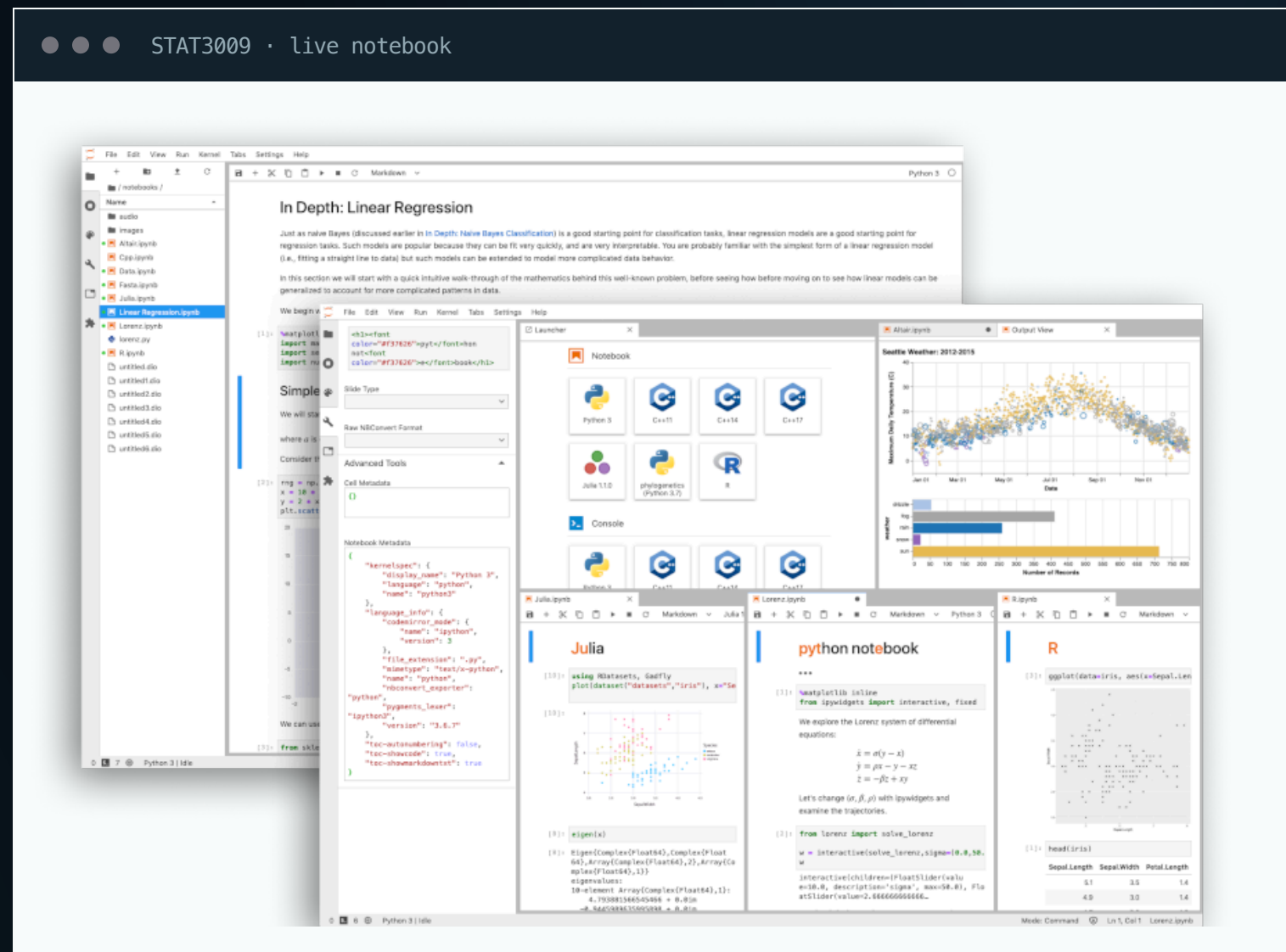
Import packages, define objects, run models, and create output.

### MARKDOWN

## Explain

Write headings, equations, links, interpretation, and task status.

Open the course notebook in Colab ↗



Source: Jupyter Project



# Use Colab on the web or in VS Code

SAME NOTEBOOK · SAME COLAB COMPUTE

## COLAB WEB

### Browser first

1. Open the course notebook below
2. Sign in with Google
3. Save a copy in your Drive
4. Run a cell with **Shift + Enter**

Open the course notebook ↗

## COLAB FOR VS CODE

### Editor first

1. Install the official **Colab** extension
2. Download the course notebook as `.ipynb`
3. Open it in VS Code
4. Choose **Select Kernel** → **Colab**
5. Select **Auto Connect** and sign in

Install the VS Code extension ↗

In Colab: **File** → **Download** → **Download .ipynb** gives you a local notebook file.

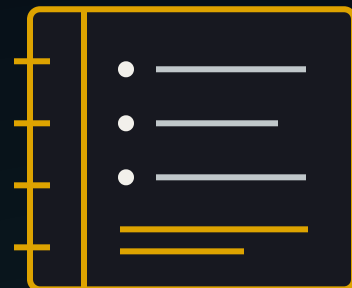


# Notebook, kernel, and Python environment

AN ANALOGY: MIXING A GLASS OF SUGAR WATER

## Notebook

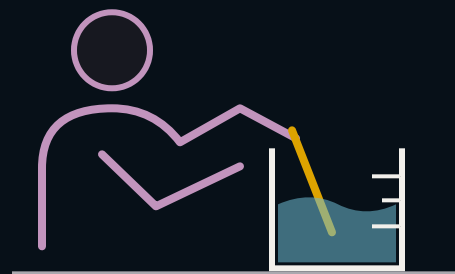
Instructions & experiment log



The `.ipynb` file stores code, explanations, and saved results.

## Kernel

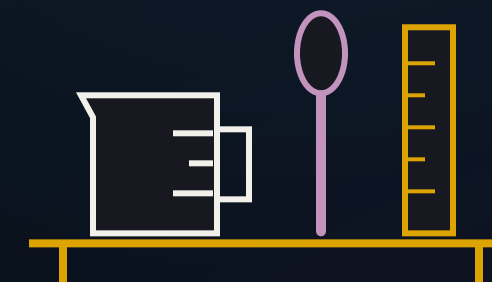
The person doing the experiment



Runs the code and holds variables in memory, like the current mixture.

## Python environment

Workbench & available tools



Provides the Python interpreter and installed packages the kernel uses.

## Restart the kernel

Start again with an empty cup. Variables disappear. Saved notebook contents remain.

## `%pip install pandas`

Add a tool to the active workbench. The package goes into the kernel's environment.



# A notebook remembers execution—not visual order

## THE KERNEL HOLDS INVISIBLE STATE

01 · RUN CELL `discount = 0.10`

kernel memory → **discount = 0.10**

02 · EDIT,  
BUT DO NOT  
RUN

`discount = 0.20`

screen → **0.20** · memory → **0.10**

03 · RUN THE  
NEXT CELL

`100 * (1 - discount)`

output → **90.0**, not 80.0

## ON SCREEN

### Saved cells

What the notebook currently displays.

## IN MEMORY

### Executed state

What the kernel will actually use.

## BEFORE SHARING OR SUBMITTING

### Restart session → Run all → Verify outputs

A fresh Colab runtime may also require the package-setup cells again.



# Markdown turns plain text into structure

## WHAT YOU TYPE

```
## Preparation
```

```
**Goal:** verify the software setup.
```

- [x] install the packages
- [ ] finish the practice

```
[Python docs](https://docs.python.org/)
```

```
Inline code: `print(version)`
```

```
Equation: $y = \beta_0 + \beta_1 x$
```

## WHAT THE NOTEBOOK RENDERS

### Preparation

**Goal:** verify the software setup.

- ☒ install the packages
- ☐ finish the practice

[Python docs](#)

Inline code: `print(version)`

$$y = \beta_0 + \beta_1 x$$



# Markdown also handles technical notation

```
### Prediction error
```

```
Use `predict(...)` to obtain predictions.
```

```
$$
\operatorname{RMSE}
= \sqrt{\frac{1}{n} \sum_{i=1}^n
(y_i - \hat{y}_i)^2}
$$
```

| symbol | meaning          |
|--------|------------------|
| y      | observed rating  |
| y_hat  | predicted rating |

## inline code

wrap a short expression in backticks

## display math

wrap an equation in double dollar signs

## pipe table

use vertical bars to separate columns

## blank lines

separate blocks and prevent rendering surprises



# Python refresher

Review the small set of patterns we will use from the first lab onward.



04



# Packages extend what Python can do

```
1 %pip install -q numpy pandas seaborn rehline
2
3 import numpy as np
4 import pandas as pd
5 import seaborn as sns
6 import rehline
7
8 print("NumPy:", np.__version__)
9 print("Pandas:", pd.__version__)
```

---

## **%pip install**

installs into the current notebook environment

---

## **import**

loads a package in the current session

---

## **as np · as pd**

creates conventional short aliases

---

## **seaborn · rehline**

match the dependencies listed in the combined course notebook

---



# Assignment connects a name to an object

```
1 course = "STAT3009"
2 week = 1
3 threshold = 3.5
4 ready = True
5
6 # inspect and compare
7 print(type(course))
8 print(week == 1)
```

---

**name = value**

assigns an object to a name

---

**str · int · float · bool**

are common scalar types

---

**==**

compares two values

---

**#**

starts a comment

---



# Expressions transform values

```
1 score = 82
2 bonus = 5
3
4 adjusted = score + bonus
5 passed = adjusted >= 60
6 in_range = 0 <= adjusted <= 100
7
8 message = f"adjusted score: {adjusted}"
9
10 print(message)
11 print(passed, in_range)
```

---

`+` `*` `-` `.` `*` `.` `/`

perform arithmetic operations

---

`>=` `.` `<=` `.` `==` `.` `!=`

produce Boolean values

---

**chained comparison**

expresses a range naturally

---

**`f"{name}"`**

inserts an expression into text

---



# Lists and dictionaries organize values

```
1 ratings = [4, 5, 3, 4]
2 student = {"name": "Ada", "ready": True}
3
4 print(ratings[0])      # first item
5 print(ratings[-1])     # last item
6 print(ratings[1:3])    # positions 1 and 2
7 print(student["name"]) # look up a key
```

---

## list

ordered values indexed by position

---

## dict

key-value pairs indexed by key

---

## [start:stop]

includes start, excludes stop

---

## -1

refers to the last item

---



# A for loop visits each value once

```
1 liked = []
2
3 for rating in ratings:
4     if rating >= 4:
5         liked.append(rating)
6
7 for index, rating in enumerate(ratings):
8     print(index, rating)
```

---

## **for item in values**

iterates over a sequence

---

## **if condition**

runs a block only when true

---

## **.append(...)**

adds one item to a list

---

## **enumerate(...)**

provides both position and value

---



# Indentation is part of Python syntax

```
1 for rating in ratings:
2     if rating >= 4:
3         label = "liked"
4     else:
5         label = "other"
6     print(label)
7
8 print("finished")
```

:

starts an indented block

## 4 spaces

is the recommended indentation

## same level

means the same block

## dedent

leaves the current block

Do not mix tabs and spaces. Python uses indentation to decide which statements belong together.



# Functions package repeated logic

```
1 def select_liked(values, threshold=4):  
2     liked = []  
3     for value in values:  
4         if value >= threshold:  
5             liked.append(value)  
6     return liked  
7  
8 select_liked(ratings)
```

---

**def name(...):**

defines a function

---

**parameters**

receive inputs

---

**local names**

exist inside the function

---

**return**

sends one result back

---



# NumPy arrays have shape and dtype

```
1 ratings = np.array([4, 5, 3, 4], dtype=float)
2
3 print("ndim:", ratings.ndim)
4 print("shape:", ratings.shape)
5 print("size:", ratings.size)
6 print("dtype:", ratings.dtype)
7
8 matrix = np.arange(12).reshape(3, 4)
9 print("matrix shape:", matrix.shape)
```

---

## **ndarray**

stores homogeneous values efficiently

---

## **ndim**

counts the number of axes

---

## **shape · size**

describe dimensions and element count

---

## **dtype**

records the common value type

---



# Axis tells NumPy which direction to summarize

```
1 matrix = np.arange(12).reshape(3, 4)
2
3 print(matrix.mean())           # one overall mean
4 print(matrix.mean(axis=0))    # one mean per column
5 print(matrix.mean(axis=1))    # one mean per row
6
7 column_totals = matrix.sum(axis=0)
8 row_totals = matrix.sum(axis=1)
9 print(column_totals, row_totals)
```

---

## **axis omitted**

summarizes all elements

---

## **axis=0**

collapses rows; keeps columns

---

## **axis=1**

collapses columns; keeps rows

---

## **check shape**

predicts the size of the result

---



# Boolean masks filter NumPy arrays

```
1 ratings = np.array([4, 5, 3, 4, 2])
2
3 mask = ratings >= 4
4 liked = ratings[mask]
5
6 middle = ratings[
7     (ratings >= 3) & (ratings <= 4)
8 ]
9 share_liked = mask.mean()
10
11 print(mask, liked, middle, share_liked, sep="\n")
```

---

## comparison

creates an array of True and False

---

## array[mask]

keeps values where the mask is True

---

## & · |

combine array conditions element by element

---

## mask.mean()

computes the share of True values

---



# Pandas adds labels to tabular objects

```
1 df = pd.DataFrame({
2     "student": ["A", "B", "C"],
3     "score": [82, 95, 74],
4 })
5
6 selected = df[["student", "score"]]
7 high_scores = df.loc[df["score"] >= 80]
8 df["centered"] = (
9     df["score"] - df["score"].mean()
10 )
11
12 display(selected, high_scores, df)
```

---

## DataFrame

a labeled table

---

## columns

select named variables

---

## .loc

filters rows explicitly

---

## assignment

creates or updates a column

---



# Pandas can sort and summarize groups

```
1 df["tutorial"] = ["A", "A", "B"]
2
3 ordered = df.sort_values(
4     "score", ascending=False
5 )
6
7 summary = (
8     df.groupby("tutorial")
9         .agg(n=("score", "size"),
10             mean_score=("score", "mean"))
11         .reset_index()
12 )
13
14 display(ordered, summary)
```

---

## **.sort\_values(...)**

orders rows by one or more columns

---

## **.groupby(...)**

splits rows into meaningful groups

---

## **.agg(...)**

calculates several summaries at once

---

## **method chain**

reads as a sequence of transformations

---



# Missing values require an explicit decision

```
1 scores = pd.DataFrame({
2     "student": ["A", "B", "C", "D"],
3     "score": [82, None, 74, 91],
4 })
5
6 print(scores.isna().sum())
7 complete = scores.dropna(subset=["score"])
8 scores["score"] = scores["score"].fillna(
9     scores["score"].median()
10 )
11
12 display(complete, scores)
```

---

**.isna()**

locates missing entries

---

**.dropna(...)**

removes rows under a stated rule

---

**.fillna(...)**

replaces missing entries

---

**document the choice**

because it changes the analysis

---



# Recommendation setup

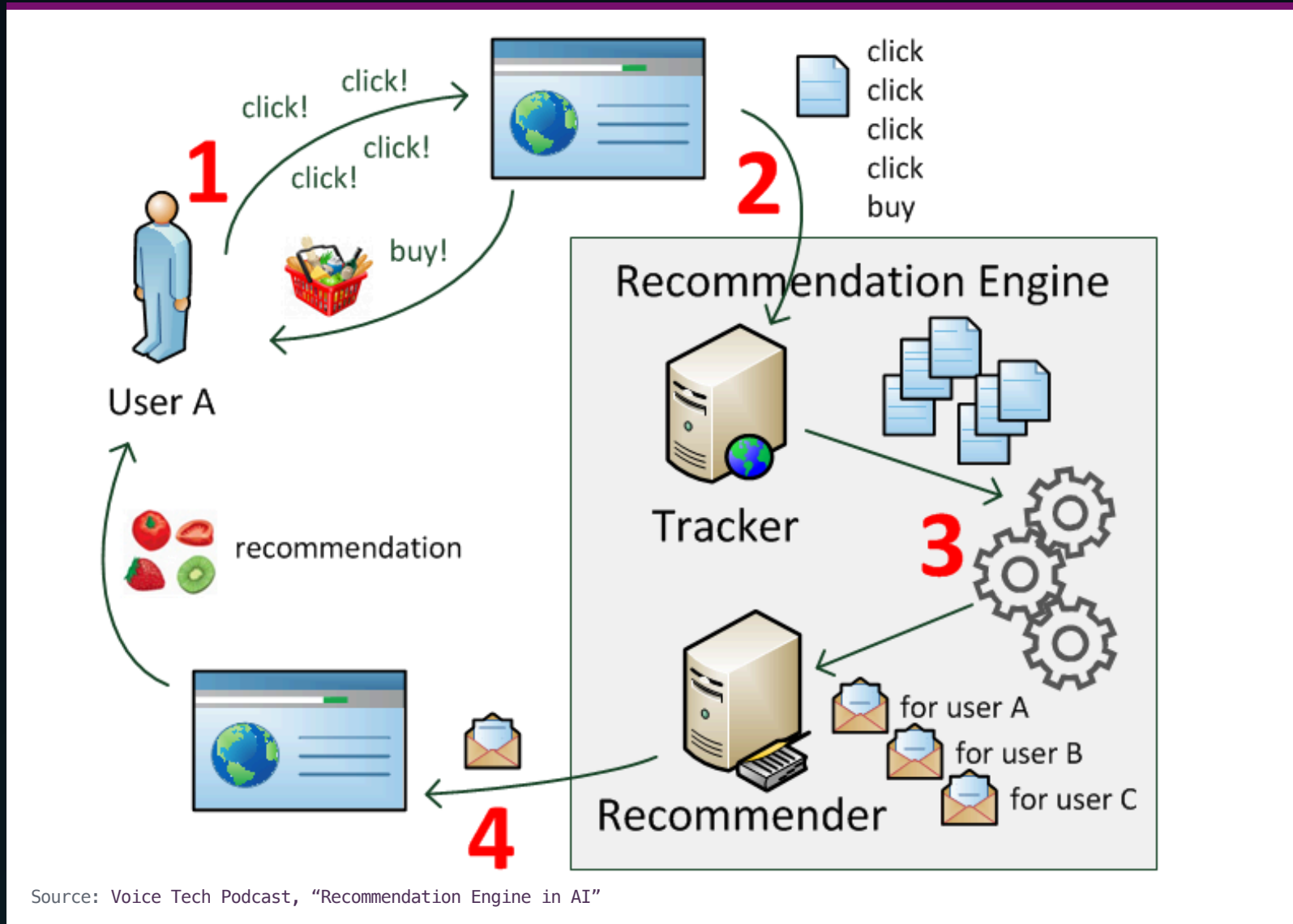
Turn observed user-item interactions into predictions for unseen pairs.



05



# The recommendation feedback loop



A recommender learns from actions produced by earlier recommendations.

- 1 Observe behavior**  
Clicks, purchases, and ratings reveal user preferences.
- 2 Record events**  
The tracker stores interactions for later learning.
- 3 Learn preferences**  
The model turns interaction history into scores.
- 4 Serve recommendations**  
The system returns selected items to each user.

New actions become the next round of training data.

# Recommendation serving and learning

**ONLINE**  
Each request

## Retrieve

Reduce the catalog to a candidate pool

→

## Rank

Score the smaller pool with a richer model

→

## Re-rank + display

Apply eligibility and diversity constraints

Outcomes become training data ↓

Updated models return to serving ↑

**OFFLINE**  
Periodic updates

## Prepare data

Log events and construct features

→

## Train + validate

Fit models and compare their predictions

→

## Deploy + monitor

Publish models and check service quality

Training and serving must use consistent feature definitions.



# This course focuses on rating prediction

## Data

Sparse user-item ratings

Identify the observed feedback and the missing pairs.

## Models

Baselines, factorization, neural models

Learn a function that predicts a rating for a user-item pair.

## Evaluation

Prediction on held-out ratings

Compare models on validation data and report final test performance.

The predicted scores can then support candidate ranking in the wider system.



# Real datasets use different names for the same roles

THREE DATASETS · ONE ABSTRACTION

| Dataset       | Fields in user / item / rating order      | Actual record (u, i, r)              |
|---------------|-------------------------------------------|--------------------------------------|
| Netflix       | <code>user_id · movie_id · rating</code>  | <code>(1960, 670, 4)</code>          |
| CiaoDVD       | <code>userID · itemID · rating</code>     | <code>(FA8D7A, 79CBAD, 5.0)</code>   |
| Book-Crossing | <code>User-ID · ISBN · Book-Rating</code> | <code>(276726, 0155061224, 5)</code> |

$(u, i, r_{ui})$  is a modeling vocabulary: IDs identify who and what; feedback records the observed preference.



# Load the table, then inspect the rows

```

1 import pandas as pd
2
3 base = (
4     "https://raw.githubusercontent.com/"
5     "statmlben/CUHK-STAT3009/main/"
6     "dataset/netflix/"
7 )
8
9 fields = ["movie_id", "user_id", "rating"]
10 train = pd.read_csv(
11     base + "train.csv", usecols=fields
12 )
13 test = pd.read_csv(
14     base + "test.csv", usecols=fields
15 )
16 train.head()

```

`train.head()`

|   | movie_id | user_id | rating |
|---|----------|---------|--------|
| 0 | 670      | 1960    | 4      |
| 1 | 152      | 1346    | 4      |
| 2 | 1741     | 785     | 4      |
| 3 | 3032     | 686     | 5      |
| 4 | 536      | 1894    | 4      |

One row records one observed user-movie rating.



# A few checks reveal the dataset structure

```

1 # rows and columns
2 print("shape:", train.shape)
3
4 # storage types
5 display(train.dtypes)
6
7 # users and movies across both splits
8 both = pd.concat([train, test])
9 ids = ["user_id", "movie_id"]
10 display(both[ids].nunique())
11
12 # response scale
13 display(train["rating"].agg(["min", "max"]))

```

|              |                                                                       |
|--------------|-----------------------------------------------------------------------|
| SHAPE        | <b>51,161 × 3</b><br>training observations × selected fields          |
| CATEGORIES   | <b>2,000 users · 3,568 movies</b><br>unique IDs across train and test |
| RATING RANGE | <b>1-5</b><br>integer-valued explicit feedback                        |
| MODEL TABLE  | <b>2 IDs · 1 target</b><br>the columns needed for rating prediction   |



# Netflix data separates IDs from ratings

STORAGE TYPE  $\neq$  MODELING ROLE

| FIELD                 | EXAMPLE | PANDAS | MEANING                             |
|-----------------------|---------|--------|-------------------------------------|
| <code>user_id</code>  | 1960    | int64  | categorical user identifier         |
| <code>movie_id</code> | 670     | int64  | categorical item identifier         |
| <code>rating</code>   | 4       | int64  | ordered score and prediction target |

## NETFLIX

### Explicit feedback

A 1–5 star rating directly states a preference.

## CONTRAST

### Implicit feedback

A click, view, or purchase only suggests a preference.

An integer ID is a label—not a quantity. User 1960 is not “larger” than user 20.



# User and movie IDs form $X$ ; rating is $y$

```
1 features = ["user_id", "movie_id"]
2
3 X_train = train[features].to_numpy()
4 y_train = train["rating"].to_numpy()
5
6 X_test = test[features].to_numpy()
7 y_test = test["rating"].to_numpy()
```

## INPUT

**$X_{\text{train}} \cdot 51,161 \times 2$**

Each row is [user\_id, movie\_id].

## TARGET

**$y_{\text{train}} \cdot 51,161$  values**

Each value is the corresponding rating.

## PREDICTION TASK

**Two IDs  $\rightarrow$  one score**

Learn  $f_{\theta}(u, i) \rightarrow \hat{r}_{ui}$ .



# Most user-movie pairs are unobserved

|        | Movie 0 | Movie 1 | Movie 2 | Movie 3 |
|--------|---------|---------|---------|---------|
| User 0 | 4       | ?       | 5       | ?       |
| User 1 | ?       | 2       | ?       | 4       |
| User 2 | 5       | ?       | 4       | ?       |
| User 3 | ?       | 3       | ?       | 5       |

## OBSERVED

$$(\mathbf{u}, \mathbf{i}) \in \Omega$$

We know  $r_{ui}$ . Split observed ratings before model selection.

## UNKNOWN

$$(\mathbf{u}, \mathbf{i}) \notin \Omega$$

We estimate the missing preference as  $\hat{r}_{ui}$ .

## Missing $\neq$ zero.

A blank cell means “not observed,” not “disliked.”

# New pairs and new IDs are different problems

| Test case                  | What training provides                            | In this Netflix split                      |
|----------------------------|---------------------------------------------------|--------------------------------------------|
| <b>Known IDs, new pair</b> | History for both IDs, but no rating for this pair | Use the learned user and movie information |
| <b>New user</b>            | No training history for this user                 | 74 test users are absent from training     |
| <b>New movie</b>           | No training ratings for this movie                | 584 test movies are absent from training   |

For an unseen ID, define a fallback such as the training-set mean. Later models can use side information.



# Three minutes: make the data contract explicit

```

1 first = train.iloc[0][
2     ["user_id", "movie_id", "rating"]
3 ]
4
5 features = ["user_id", "movie_id"]
6 X = train[features].to_numpy()
7 y = train["rating"].to_numpy()
8
9 display(first)
10 print("X:", X.shape, "y:", y.shape)

```

01 Write the first observation as  $(u, i, r_{ui})$ .

02 Report the shapes of **X** and **y**.

03 Explain why an integer ID is a **label**, not a continuous measurement.

04 Complete the sentence: "A missing rating means \_\_\_\_, not zero."

Run it · compare with a neighbor ·  
explain one answer aloud



# Training, validation, and test ratings

## Training

$\Omega^{\text{tr}}$

Fit model parameters.

Use the fitting portion of `train.csv`.

## Validation

$\Omega^{\text{val}}$

Choose the method and its settings.

Reserve a portion of `train.csv`.

## Test

$\Omega^{\text{te}}$

Report final predictive performance.

Keep `test.csv` untouched during model selection.

After choosing settings, refit on all of `train.csv`, then evaluate once on `test.csv`.



# The task is to predict held-out ratings accurately

## INPUT

$(u, i)$

a candidate user-movie pair



## MODEL

$f_{\theta}$

fitted using training ratings  $\Omega^{\text{tr}}$



## PREDICTION

$\hat{r}_{ui}$

an estimated preference score

## EVALUATE ON HELD-OUT PAIRS

Compare predictions with known ratings in  $\Omega^{\text{te}}$ .

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega^{\text{te}}|} \sum_{(u,i) \in \Omega^{\text{te}}} (r_{ui} - \hat{r}_{ui})^2}$$



# Training error and prediction on new ratings

This code measures training fit. What evidence would support prediction on new ratings?

```
1 # X_train: user and movie IDs
2 # y_train: observed ratings
3 model.fit(X_train, y_train)
4
5 pred = model.predict(X_train)
6 rmse = np.sqrt(
7     np.mean((y_train - pred) ** 2)
8 )
9 print(rmse)
```

## Read the code

Fitting and evaluation use the same observations.

## Interpret the number

This is training RMSE, measuring fit to the training ratings.

## Identify the missing evidence

Training RMSE alone does not establish accuracy on new ratings.

## Change the experiment

Use validation ratings to choose settings. Keep test ratings for the final report.

Coding reveals which data was used. Mathematics explains what the result can support.



# Ranking unrated candidates by predicted score

RATING PREDICTION → CANDIDATE SCORES → RANKED LIST

01 · PREDICT

For user 1960

$$\hat{r}_{ui} = f_{\theta}(u, i)$$

Score candidate movies that this user has not rated.



02 · SCORE CANDIDATES

Illustrative predictions

|            |     |
|------------|-----|
| Movie 13   | 4.2 |
| Movie 14   | 4.8 |
| Movie 3032 | 3.9 |



03 · SORT DESCENDING

Recommendation ranking

|   |            |     |
|---|------------|-----|
| 1 | Movie 14   | 4.8 |
| 2 | Movie 13   | 4.2 |
| 3 | Movie 3032 | 3.9 |

COURSE FOCUS

**Learn the scoring function**

Estimate ratings accurately from sparse observations and evaluate with held-out RMSE.

SYSTEM USE

**Turn scores into an ordered list**

Rank candidates by predicted preference, then apply later serving constraints.

Observed ratings → rating model → predicted scores → ranked list → user actions → new observations



# Offline accuracy is a gate—not the final outcome

MODEL → SYSTEM → PRODUCT

## 01 · MODEL

### Held-out RMSE

Can predicted ratings generalize to unseen user-item pairs?

## 02 · SYSTEM

### Latency · reliability · coverage

Can the service respond quickly and consistently for real users?

## 03 · PRODUCT

### Satisfaction · engagement · retention

Does the full experience improve the intended user outcome?

OFFLINE VALIDATION → ONLINE EXPERIMENT → MONITOR AFTER LAUNCH

The objective shapes the behavior of the system. A better model metric does not automatically imply a better product.



# Ready for the first lab?

- 
- 01 Bring a laptop
  - 02 Open the notebook
  - 03 Run the setup cell
  - 04 Trace ratings → scores → ranked list → new feedback

[bendai.org/CUHK-STAT3009](https://bendai.org/CUHK-STAT3009) ↗

